

TP BTS SIO [Option Slam]

Développement d'une application de quiz interactif

► Objectifs

- concevoir une interface web dynamique
- manipuler le DOM en JavaScript
- comprendre le fonctionnement client / serveur
- créer une API simple avec Node.js
- interagir avec une base de données SQL
- implémenter une logique métier (score, validation, enchaînement)

► Contexte

Un centre de formation souhaite proposer à ses apprenants un outil de quiz en ligne permettant de s'entraîner sous forme de QCM.

L'application devra permettre :

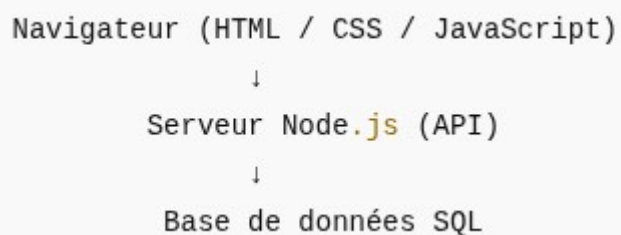
- d'afficher une série de questions
- de sélectionner une réponse
- de calculer un score final
- d'afficher un résultat à la fin du quiz

L'application fonctionnera en local, sans authentification.

Chaque question comportera plusieurs réponses possibles, dont une seule correcte.

► Architecture attendue

L'application devra respecter l'architecture suivante :



Les données (questions et réponses) devront être stockées dans une base de données.

➤ Travail à réaliser

1 Interface utilisateur

Vous devrez concevoir une interface web comprenant au minimum :

- un titre,
- une zone d'affichage de la question,
- des éléments permettant de sélectionner une réponse,
- un bouton de validation,
- une zone d'affichage du score final.

L'interface devra être claire, lisible et fonctionnelle.

2 Logique côté client

Le code JavaScript côté client devra permettre :

- l'affichage dynamique des questions et des réponses,
- la récupération du choix de l'utilisateur,
- la vérification de la réponse,
- le calcul et la mise à jour du score,
- le passage automatique à la question suivante.

3 Serveur applicatif

Un serveur Node.js devra être mis en place afin de :

- fournir les questions au client,
- recevoir les réponses de l'utilisateur,
- centraliser la logique d'accès aux données.
- Les échanges entre le client et le serveur devront se faire via une API.

4 Base de données

Une base de données SQL devra être créée afin de stocker :

- les questions,
- les réponses associées,
- l'indication de la réponse correcte.

Le modèle de données devra être cohérent et permettre l'exploitation par l'API.

➤ Contraintes techniques

- L'application devra fonctionner en local.
- L'utilisation de frameworks frontend (React, Vue, Angular...) est interdite.
- Le code devra être structuré, lisible et commenté.

Les données ne devront pas être codées en dur dans la version finale.

➤ Bonus (facultatif)

Les fonctionnalités suivantes pourront être ajoutées :

- chronomètre par question,
- affichage d'un message personnalisé selon le score,
- mélange aléatoire des questions,
- bouton permettant de rejouer le quiz.

➤ Livrables attendus

À la fin du TP, vous devrez rendre :

- le dossier complet du projet (frontend et backend),
- une application fonctionnelle,
- un court fichier README expliquant le fonctionnement général (facultatif mais recommandé).

➤ Notes

Le TP est ambitieux, vous avez le droit de travailler en groupe ou d'utiliser des outils online pour vous accompagner.

Certains éléments seront nouveaux, je vous demande de prendre le temps de les comprendre, n'hésitez pas à vous renseigner sur

- l'implémentation d'un serveur Node (la librairie Express est tolérée)
- la gestion de la liaison entre la table Questions et Reponses
- la methode fetch qui permet des récupérer des éléments d'une base de données.

Le livrable est à m'envoyer jeudi 08 Janvier en fin de journée à l'adresse suivante :
nicolas.scolari@ynov.com

Planning indicatif du TP – Application de quiz

1. Prise en main du sujet – Analyse (≈ 45 min)

Objectifs

- Comprendre le fonctionnement global du quiz
- Identifier les fonctionnalités attendues
- Visualiser l'architecture de l'application

Travail conseillé

- Lire entièrement l'énoncé
- Réfléchir à la logique d'enchaînement des questions
- Lister les données nécessaires (questions, réponses, score)

2. Conception de l'interface HTML / CSS (≈ 1h30)

Objectifs

- Mettre en place une interface claire et structurée
- Préparer les zones qui seront manipulées en JavaScript

Travail conseillé

- Créer la structure HTML complète
- Appliquer un style simple et lisible

Vérifier l'affichage dans le navigateur

À la fin de cette étape, la page doit être visuellement complète, même si elle n'est pas encore fonctionnelle.

3. Logique JavaScript côté client (≈ 1h30)

Objectifs

- Afficher dynamiquement une question
- Gérer les réponses de l'utilisateur
- Mettre en place le score et l'enchaînement

Travail conseillé

- Manipuler le DOM
- Gérer les événements utilisateur
- Tester la logique avec des données temporaires

4. Mise en place du serveur Node.js (≈ 1h30)

Objectifs

- Comprendre le rôle du serveur
- Créer une API simple

Travail conseillé

- Initialiser le projet Node.js
- Créer les routes nécessaires
- Tester les réponses de l'API

À ce stade, le serveur peut fonctionner sans base de données définitive.

5. Base de données SQL (≈ 1h)

Objectifs

- Stocker les questions et les réponses
- Comprendre la relation entre les données

Travail conseillé

- Créer les tables nécessaires
- Insérer quelques questions de test

Vérifier les requêtes SQL

6. Connexion complète & finalisation (≈ 45 min)

Objectifs

- Relier le frontend, l'API et la base de données
- Tester le quiz du début à la fin

Travail conseillé

- Charger les questions depuis la base
- Vérifier le calcul du score

Corriger les bugs éventuels

7. Temps restant / Avance

Si le temps le permet :

- améliorer l'interface
- ajouter un message de fin personnalisé
- implémenter une fonctionnalité bonus
- nettoyer et commenter le code

8. Important

Ce planning est indicatif :

- certains avanceront plus vite sur certaines phases,
- d'autres auront besoin de plus de temps ailleurs.

La priorité est le bon fonctionnement global et la compréhension, pas de "finir à tout prix".